

sich entwickeln ja neben dem Plugin, ich habe z.B. ein Plugin GoPath, nennt sich das. GoPath kommt dann mit dem und den kann man das also das kann auch Truncated Paths, also abgeschnittene Pfade, die in irgendein Buffer sind, dann k

Ein **zentraler Dateisystem-Cache-Daemon**, auf den Neovim-Plugins über IPC (z. B. Unix Domain Sockets oder RPC) zugreifen, ist architektonisch extrem elegant. Er entkoppelt rechenintensive/I/O-lastige Aufgaben vollständig von der Single-Threaded-Lua-Event-Loop in Neovim.

Ob sich das performance- und architekturmäßig lohnt, hängt von zwei Faktoren ab: **I/O-Overhead durch IPC** vs. **Vermeidung von redundantem FS-Scanning / GC-Druck in Lua**.

Vor- und Nachteile der Cache-Service-Architektur

- **Kein Blocking der Neovim-UI:** Selbst bei riesigen Projekten (`node_modules` , `target` , Riesen-Monorepos) blockiert das initiale Scannen oder Re-Indexieren niemals die Event-Loop von Neovim.
- **Single Source of Truth:** `GoPath` , dein File-Tree-Wrapper und andere Plugins teilen sich denselben Speicherzustand. Ändert sich eine Datei, wird der Cache nur einmal aktualisiert.
- **Weniger GC-Druck in LuaJIT:** Tausende String-Pfade und Tabellen-Strukturen verbrauchen in LuaJIT RAM und belasten den Garbage Collector. Im externen Daemon (z. B. in Go oder Rust geschrieben) fällt dieser Overhead im Vim-Prozess komplett weg.
- **IPC-Latency Overhead:** Jeder Zugriff benötigt Interprozesskommunikation. Bei Sockets liegt die Latenz typischerweise unter **0.1–0.5 ms**. Wenn du pro Tastendruck 1.000 Pfade einzeln abfragst, summiert sich das. Bei Batch-Abfragen (oder Subscriptions) ist das vernachlässigbar.
- **Prozess-Lifecycle-Management:** Neovim muss den Daemon zuverlässig starten, überwachen und beim Beenden (oder Absturz) aufräumen.

Was bringt es für die Performance?

Die Performance-Bewertung unterscheidet sich stark nach Anwendungsfall:

Szenario	Evaluierung	Grund
Startup-Zeit von Neovim	Massiver Gewinn	Neovim startet sofort; der Daemon liest das Dateisystem asynchron im Hintergrund ein.
Speicherverbrauch in Neovim	Deutlicher Gewinn	Tausende Pfad-Strings belasten nicht den LuaJIT-Heap.
Fuzzy-Matching / Abfrage	Minimal langsamer (einzeln) / Gleich schnell (in Batches)	Ein direkter Lua-Table-Lookup dauert Nanosekunden; IPC dauert Mikroskunden. Solange nicht hunderte Synch-Calls pro Frame gemacht werden, merkt der Nutzer keinen Unterschied.
Dateisystem-Invalidierung	Sehr großer Gewinn	Nur ein Prozess lauscht auf File-System-Events (<code>inotify</code> / <code>fsevents</code>), statt dass jedes Plugin eigene Watcher startet.
...		

Empfohlener Architektur-Ansatz

Wenn du diesen Weg gehst, profitiert die Performance vor allem von folgendem Setup:

- Kommunikation via Unix Domain Socket (oder Named Pipes unter Windows):**
Lokal extrem schnell. Neovim bietet mit `vim.uv.new_pipe()` (Libuv) nativer Support für asynchrone Socket-Kommunikation direkt aus Lua.
- Prozess-Sprache:**
Ein kompiliertes Binary (Go, Rust oder C) ist ideal. Go eignet sich hervorragend für schnelle Concurrent FS-Walks via Goroutines; Rust bietet maximale Kontrolle über Speicher und Inotify/fsevents-Anbindungen.
- Event-driven Invalidation:**
Der Daemon registriert FS-Watcher auf dem Projektverzeichnis und hält den Cache im Hintergrund automatisch fresh.

4. **Batching / Stream-APIs:**

Statt für 50 Pfad-Snippets 50 IPC-Requests zu schicken, schicke eine Liste von Truncated Paths im Batch rüber und erhalte ein JSON/MessagePack-Array mit den aufgelösten Pfaden zurück.

Fazit: Ja, das Konzept lohnt sich sehr – weniger wegen der reinen Lookups-Geschwindigkeit (wo In-Memory-Lua kaum zu schlagen ist), sondern vor allem für die **Entlastung der Neovim Event-Loop, Sofort-Startzeiten** und die **Vermeidung von redundantem FS-Scanning** über mehrere Plugins hinweg.

Das könnte auch eine Beobachtung von mir fixen bei der immer beim nvim start ich sofort curdur habe aber nach 500ms freier der Cursor für 1-3 Sekunden ein - das könnte der FS Cache von gopath sein der bei jeden Start startet, wrsch noch nicht async ist

Das trifft den Nagel ziemlich genau auf den Kopf. Ein Einfrieren des Cursors **1 bis 3 Sekunden** nach dem Start ist das klassische Symptom für einen synchronen (blockierenden) I/O- oder Rechenprozess in der Lua-Event-Loop von Neovim.

Der zeitliche Ablauf passt exakt zu deiner Vermutung:

1. **Start (0–500 ms):** Neovim lädt die UI und deine Plugins. Da der Cursor sofort da ist, läuft das Init-Skript flüssig durch.
2. **Hintergrund-Delay (~500 ms):** Ein Timer, ein `VimEnter`-Autocmd oder ein verzögerter Event-Callback löst den Dateisystem-Scan von `GoPath` aus.
3. **Der Hänger (1–3 Sekunden):** `GoPath` traversiert das Dateisystem über synchrone System-Calls (wie `uv.fs_scandir` ohne Callback oder `vim.fn.glob` / `lfs`). Da LuaJIT in Neovim **single-threaded** arbeitet, blockiert jede synchrone Dateisystem-Abfrage die gesamte Event-Loop – Eingaben, Cursor-Rendering und UI-Updates stehen komplett still, bis der Tree-Walk abgeschlossen ist.

Verifizierung: Ist es wirklich GoPath?

Du kannst das schnell überprüfen, ohne Code umzubauen:

- **GoPath temporär deaktivieren:** Wenn der Lag beim Start weg ist, hast du den Übeltäter.
- **Profiler nutzen:** Starte Neovim mit dem integrierten Profiler:

Vim Script



```
:profile start profile.log
:profile func *
:profile file *
" kurz warten bis der Lag vorbei ist, dann:
:profile pause
```

In `profile.log` siehst du sofort, welche GoPath-Funktion die Sekunden frisst.

Was dir der eigene Daemon-Prozess hier bringt

Wenn du den FS-Scan in einen separaten Prozess (Daemon) auslagerst, verändert sich das Verhalten grundlegend:

- **Null UI-Blocking:** Der Daemon führt den Dateisystem-Walk in einem eigenen Betriebssystem-Prozess aus. Deine Neovim-Instanz kriegt davon überhaupt nichts mit.
- **Sofortige Nutzbarkeit:** Du kannst ab der ersten Millisekunde tippen. Falls du innerhalb der ersten 1–2 Sekunden einen Pfad auflösen willst und der Cache noch nicht ganz fertig eingelesen ist, wartest du höchstens asynchron auf die Antwort – aber Neovim friert niemals ein.
- **Kein LuaJIT-Overhead:** Das Traversieren von Zehntausenden Ordnern erzeugt in Lua massenhaft temporäre String- und Table-Objekte, die wiederum den Garbage Collector triggern. Ein kompilierter Daemon (z. B. in Go oder Rust) erledigt den Tree-Walk um ein Vielfaches schneller und ressourcenschonender.

Wenn du `GoPath` und deine anderen Plugins auf diesen Daemon umstellst, ist dieses lästige Cursor-Einfrieren beim Start endgültig Geschichte.